



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impossible for complex environments like Chess because it would need to store an action for every possible sequence of percepts. As the agent's lifetime increases, the number of possible percept histories grows exponentially. This causes a combinatorial explosion, where the required memory and computation become impossibly large.

For example, in Chess, the number of possible board states and move sequences is enormous, so creating a complete lookup table containing the best action for every situation would require unrealistic amounts of storage.

Therefore, instead of Table-Driven Agents, AI systems use Agent Programs with rules, models, search, and learning techniques to make decisions efficiently.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Rule 1 (if percept['food_here']):

Condition: The sensory percept checks if the agent is currently standing directly on top of a food pellet ('food_here' == True).

Action: The agent returns the action 'Up' to interact with the environment.

Rule 2 (elif percept['wall_ahead']):

Condition: The sensory percept detects an obstacle blocking the path directly ahead ('wall_ahead' == True).

Action: The agent executes the fallback evasion action 'Left' to turn away from the wall.

Rule 3 (else):

Condition: The default catch-all condition when neither of the above triggers apply (i.e., path is clear and no food is immediately present).

Action: The agent defaults to moving 'Up'.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Partial Observability:

The agent does not have a complete or persistent map of the grid. Its sensory input is limited to immediate, localized percepts (percept['food_here'] and percept['wall_ahead']). Because it cannot see the global layout of walls, open paths, or its overall trajectory, it cannot deduce a strategic route out of tight spots or corners. It only reacts to the exact micro-state it occupies right now.

Lack of Percept History (No Internal State):

A simple reflex agent has no memory. It does not remember what action it took in the previous time step, how many times it hit the same wall, or which cells it has already visited.

Every decision is evaluated in isolation based solely on the current input dictionary.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Sensor Model- The sensor model is responsible for mapping the current environmental sensory input into internal representations. this is handled via the incoming percept dictionary (percept['food_here'] and percept['wall_ahead']), combined with internal memory variables like self.visited_cells and self.last_action.

Transition Model- A proper transition model should track how the world evolves and how the agent's actions affect the state (e.g., predicting how an action like 'Left' or 'Up' alters the agent's (x, y) coordinates).

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING